SIMATS ENGINEERING

ASSESSMENT TOOL 1: Expert System Development Project

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.*(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 1

Annexure A

Expert System Development Project

Code of Conduct:

*I **RASHMITHA SENTHIL KUMAR (192525034)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: RASHMITHA .S

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	**100%**	

Signature of the Faculty Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

Tool: SWI-Prolog

Usage of Prolog: Students shall use **Prolog** to develop a rule-based expert system by representing domain knowledge as **facts and production rules**, and implementing reasoning through **forward and backward chaining**. The project should demonstrate Prolog's **declarative/non-procedural nature**, logical inference, rule matching, and automated conclusion generation.

S.No.	Scenario-Based Expert System Development Question
1	**Medical Diagnosis Expert System:** Develop a rule-based expert system that identifies possible diseases based on symptoms, patient observations, and basic clinical information. Represent domain knowledge using production rules and implement both forward and backward chaining to derive conclusions.
2	**Crop Disease Advisory System:** Develop an expert system that identifies possible crop diseases based on leaf symptoms, soil conditions, weather conditions, and visible plant characteristics. Construct suitable production rules and demonstrate diagnosis using forward and backward chaining.
3	**Automobile Fault Diagnosis System:** Design an expert system that identifies probable vehicle faults based on symptoms such as engine noise, starting problems, warning indicators, abnormal vibration, and reduced mileage. Implement the knowledge base using production rules and compare forward and backward reasoning.
4	**Student Academic Advisory System:** Develop an expert system that recommends suitable academic interventions based on attendance, internal marks, assignment performance, learning difficulties, and previous academic performance. Use production rules and demonstrate both forward and backward chaining.
5	**Cybersecurity Threat Diagnosis System:** Construct a rule-based expert system that identifies possible cybersecurity threats from events such as repeated login failures, unusual network traffic, suspicious file activity, and unauthorized access attempts. Implement production rules and demonstrate how forward and backward chaining arrive at a threat classification.

Report Format:

Stage	Student Activity	Expected Output
1. Domain Analysis	Identify the problem domain, entities, facts, symptoms/conditions, and possible conclusions.	Problem definition and domain model
2. Knowledge Acquisition	Collect domain knowledge from reliable sources or domain experts.	Knowledge specification
3. Knowledge Representation	Convert domain knowledge into IF–THEN production rules.	Knowledge base
4. Inference Design	Implement forward chaining and backward chaining.	Inference engine
5. Paradigm Analysis	Distinguish how the solution can be expressed using procedural and non-procedural approaches.	Paradigm comparison
6. System Development	Integrate the knowledge base, inference engine, user interface, and explanation facility.	Working expert system
7. Testing	Test the system using multiple facts and scenarios.	Test cases and outputs
8. Evaluation	Analyze correctness, coverage, consistency, and reasoning performance.	Evaluation results
9. Documentation	Prepare technical documentation and GitHub repository.	Project report + GitHub
10. Demonstration	Demonstrate the system and explain the reasoning process.	Project presentation/viva

CYBERSHIELD-ES: A RULE-BASED CYBERSECURITY THREAT DIAGNOSIS EXPERT SYSTEM USING SWI-PROLOG

CO5 ASSESSMENT TOOL -1

EXPERT SYSTEM DEVELOPMENT PROJECT

Submitted in the partial fulfilment for the Course of

MLA0104-ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

RASHMITHA SENTHIL KUMAR (192525034)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

AUGUST 2026

1. PROBLEM STATEMENT & OBJECTIVES:

Cybersecurity systems receive a large number of login, file-access and network events. Manually examining every event is difficult and may delay the identification of cyber threats. Individual security events may appear harmless, but a combination of related events can indicate a serious attack.

The main objectives of the system are:

- Identify the important entities, facts, conditions and relationships in the cybersecurity domain.
- Represent cybersecurity knowledge using production rules.
- Represent the selected problem using propositional logic.
- Model users, devices and security events using first-order logic.
- Implement the knowledge base using Prolog facts, rules and queries.
- Demonstrate forward-chaining and backward-chaining inference.
- Compare the four knowledge-representation approaches.
- Identify the most suitable implementation approach.
- Provide security recommendations based on the detected threat.

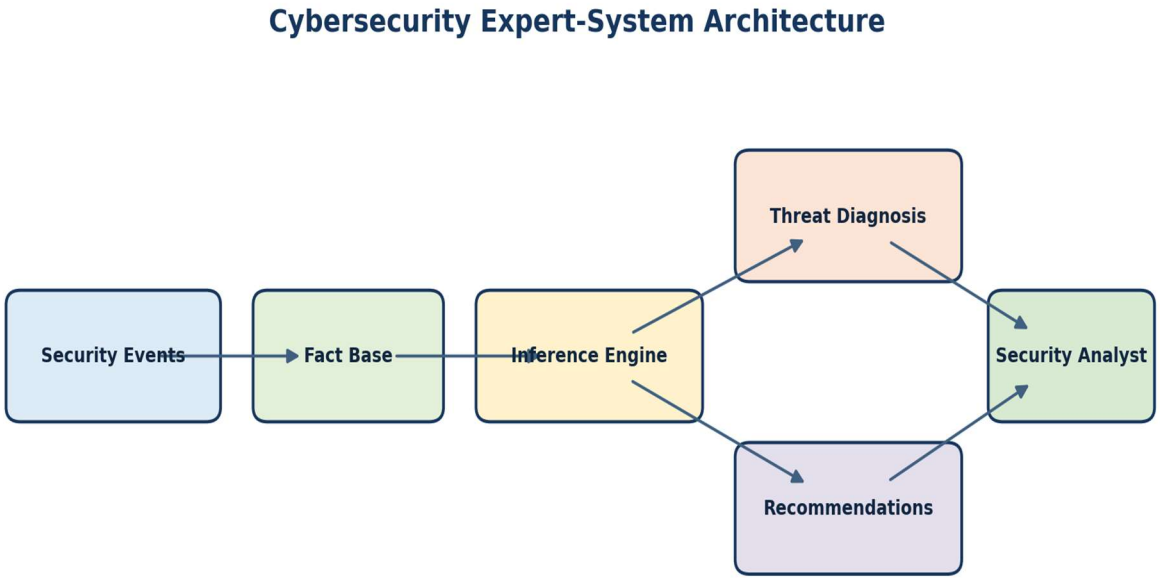

Figure 1: Architecture of the Cybersecurity Threat Detection Expert System

2. DOMAIN KNOWLEDGE & FACTS:

The cybersecurity domain consists of users, devices, login events, locations, files, network events and security alerts. The system must identify the relationships among these elements before diagnosing a threat.

The main entities are users, devices, locations, files, login events, network events and security alerts. Important attributes include username, device name, failed-login count, login location, privilege level, accessed file, network condition, threat category and recommended action.

Knowledge Acquisition:

The domain knowledge was acquired from the NIST Cybersecurity Framework 2.0, the MITRE ATT&CK Enterprise Matrix, the SWI-Prolog reference manual and standard artificial-intelligence literature. NIST supplied the incident-identification and response context; MITRE ATT&CK supplied recognizable authentication, privilege, file-access and data-transfer behaviours; and the Prolog sources guided the representation of facts, Horn-clause rules, unification and backtracking.

The sources were converted into a bounded academic knowledge specification. Repeated authentication failures represent credential-attack evidence, unusual location represents suspicious authentication context, privilege escalation represents increased account risk, sensitive-file access represents protected-resource activity, and abnormal outbound traffic represents possible data movement. The threshold of five failures is a prototype design parameter rather than a universal security standard.

Knowledge Specification:

The selected case contains the following facts:

- Alice has six failed login attempts.
- Alice logged in from an unusual location.
- Alice performed privilege escalation.
- Alice accessed the sensitive payroll database.
- Alice's workstation generated abnormal network traffic.
- Workstation ws17 belongs to Alice.

Scenario Coverage:

Four independent scenarios are included in the corrected implementation: Alice supplies all evidence for a critical threat; Charlie supplies repeated failures only and therefore supports brute force without account compromise; David supplies sensitive-file access and abnormal traffic and therefore supports data exfiltration without a critical threat; and Bob supplies insufficient evidence and remains a negative case.

A repeated-login condition occurs when the number of failed login attempts is five or more. Several failed attempts may indicate a brute-force attack. Repeated login failures combined with an unusual login location may indicate a compromised account. Privilege escalation combined with suspicious file access indicates possible privilege misuse. Suspicious file access combined with abnormal network traffic can indicate possible data exfiltration.

The model uses a fixed failed-login threshold, analyses predefined events, assumes correlated events occur within one monitoring period, and treats missing facts as unproven. Every diagnosis is an alert requiring human investigation; the prototype does not automatically block a user or device.

3. PRODUCTION RULE MODEL:

Production rules represent knowledge using an IF condition, THEN conclusion format. The following rules connect observable security evidence to diagnoses and response actions:

Rule 1: IF failed login count is five or more, THEN repeated login failures are detected.

Rule 2: IF repeated login failures are detected, THEN infer a brute-force attempt.

Rule 3: IF a user logs in from an unusual location, THEN infer a suspicious login.

Rule 4: IF a brute-force attempt and suspicious login occur for the same user, THEN infer a compromised account.

Rule 5: IF privilege escalation and suspicious file access occur, THEN infer privilege misuse.

Rule 6: IF a user accesses a sensitive file and the user's device has abnormal traffic, THEN infer possible data exfiltration.

Rule 7: IF an account is compromised and privilege escalation occurs, THEN classify it as high risk.

Rule 8: IF a high-risk account also shows possible data exfiltration, THEN infer a critical threat.

Rule 9: IF a critical threat is detected, THEN recommend isolating the device.

Rule 10: IF a critical threat is detected, THEN recommend resetting the user's credentials.

Rule 11: IF privilege misuse is detected, THEN recommend reviewing access logs.

Rule 12: IF a brute-force attempt is detected but compromise is not proven, THEN recommend multi-factor authentication.

The production-rule approach is transparent because each conclusion can be traced to a fired rule. Its main limitation is that a very large rule base can become difficult to maintain.

4. PROPOSITIONAL LOGIC MODEL:

Propositional logic represents each observation and conclusion using a symbol. Let P = at least five failed logins, Q = unusual login location, R = privilege escalation, S = sensitive-file access, T = abnormal device traffic, B = brute-force attempt, L = suspicious login, C = compromised account, E = data exfiltration, H = high risk and K = critical threat.

P -> B

Q -> L

(B AND L) -> C

(S AND T) -> E

(C AND R) -> H

(H AND E) -> K

P, Q, R, S and T are true in the selected case. Modus ponens derives B and L, then C, E and H, and finally K. Propositional logic is clear for a small case, but separate symbols are required for every user and device.

5. FIRST-ORDER LOGIC MODEL:

First-order logic represents users, devices, resources and relationships using predicates, variables and quantifiers. The principal predicates are FailedCount(u,n), UnusualLocation(u,l), Escalated(u), SuspiciousAccess(u,r), Owns(u,d), AbnormalTraffic(d), Compromised(u) and CriticalThreat(u).

For all u,n: FailedCount(u,n) AND n >= 5 -> RepeatedFailures(u)

For all u: RepeatedFailures(u) -> BruteForce(u)

For all u,l: UnusualLocation(u,l) -> SuspiciousLogin(u)

For all u: BruteForce(u) AND SuspiciousLogin(u) -> Compromised(u)

For all u,d,r: Owns(u,d) AND SuspiciousAccess(u,r) AND AbnormalTraffic(d) -> DataExfiltration(u)

For all u: Compromised(u) AND Escalated(u) -> HighRisk(u)

For all u: HighRisk(u) AND DataExfiltration(u) -> CriticalThreat(u)

First-order logic is more expressive than propositional logic because the same rules apply to different users and devices. However, implementing a complete theorem prover is more complex than writing an equivalent Prolog program.

6. PROLOG MODEL & IMPLEMENTATION:

Prolog represents the domain through facts, rules and queries. Constants begin with lowercase letters, variables begin with uppercase letters, and every clause ends with a full stop.

```
% Facts
failed_login_count(alice, 6).
unusual_location(alice, berlin).
privilege_escalation(alice).
suspicious_file_access(alice, payroll_db).
abnormal_network_traffic(ws17).
owns(alice, ws17).

repeated_login_failures(User) :-
   failed_login_count(User, Count), Count >= 5.
brute_force_attempt(User) :- repeated_login_failures(User).
suspicious_login(User) :- unusual_location(User, _).
compromised_account(User) :-
   brute_force_attempt(User), suspicious_login(User).
privilege_misuse(User) :-
   privilege_escalation(User), suspicious_file_access(User, _).
possible_data_exfiltration(User) :-
   owns(User, Device), suspicious_file_access(User, _),
   abnormal_network_traffic(Device).
```

```
high_risk_account(User) :-
    compromised_account(User), privilege_escalation(User).
critical_threat(User) :-
    high_risk_account(User), possible_data_exfiltration(User).
recommend(User, isolate_device) :- critical_threat(User).
recommend(User, reset_credentials) :- critical_threat(User).
recommend(User, review_access_logs) :- privilege_misuse(User).
recommend(User, enforce_mfa) :-
    brute_force_attempt(User), \+ compromised_account(User).
all_recommendations(User, Actions) :-
    setof(Action, recommend(User, Action), Actions).
```

Unification matches User with alice and Device with ws17. The anonymous variable (_) shows that the exact unusual location or file is not needed for some conclusions. Backtracking returns alternative recommendations when the user requests another solution.

Explicit Forward-Chaining Engine:

The corrected implementation contains a genuine data-driven fixpoint engine. It loads observable seed facts into working memory, fires every rule whose conditions are present, adds new conclusions, and repeats until no additional conclusion can be derived.

```
forward_chain(User, FinalFacts) :-
    findall(Fact, forward_seed(User, Fact), SeedFacts0),
    sort(SeedFacts0, SeedFacts),
    forward_fixpoint(SeedFacts, FinalFacts).


forward_fixpoint(CurrentFacts, FinalFacts) :-
    findall(Conclusion,
          ( forward_rule(Conclusion, Conditions),
            all_present(Conditions, CurrentFacts),
            \+ memberchk(Conclusion, CurrentFacts)
          ), NewFacts0),
    sort(NewFacts0, NewFacts),
    ( NewFacts = [] -> FinalFacts = CurrentFacts
```

```
  ; append(CurrentFacts, NewFacts, Expanded0),
    sort(Expanded0, Expanded),
    forward_fixpoint(Expanded, FinalFacts)
  ).
```

System Development and Explanation Facility:

The working system integrates the fact base, backward rules, explicit forward-chaining engine, console diagnosis, recommendation facility, recursive explanation trace and grouped test module. The diagnose/1 predicate provides the user interface, while explain_critical_threat/1 displays the proof path supporting a critical-threat conclusion.

Queries and Outputs:

```
?- repeated_login_failures(alice).
true.

?- critical_threat(alice).
true.

?- recommend(alice, Action).
Action = isolate_device ;
Action = reset_credentials ;
Action = review_access_logs.

?- critical_threat(bob).
false.
```

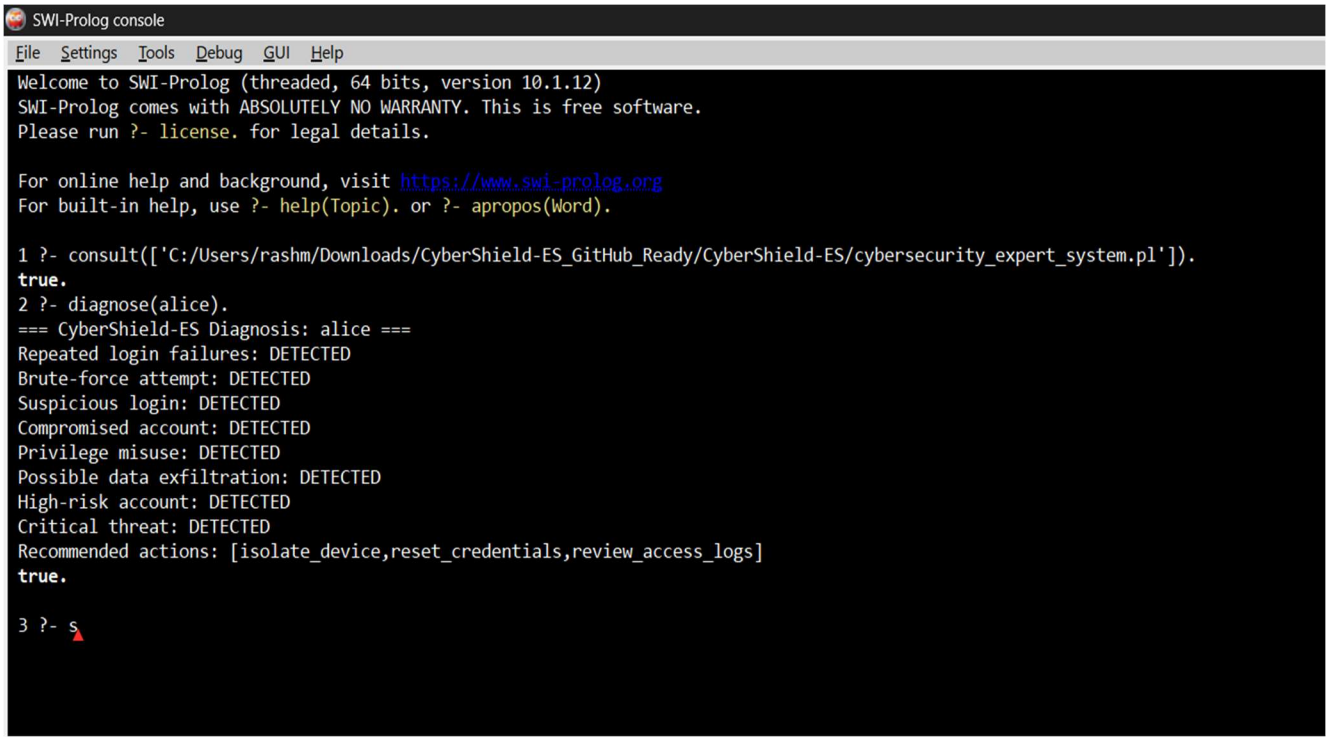

Figure 2: Real SWI-Prolog Output of the Complete CyberShield-ES Diagnosis

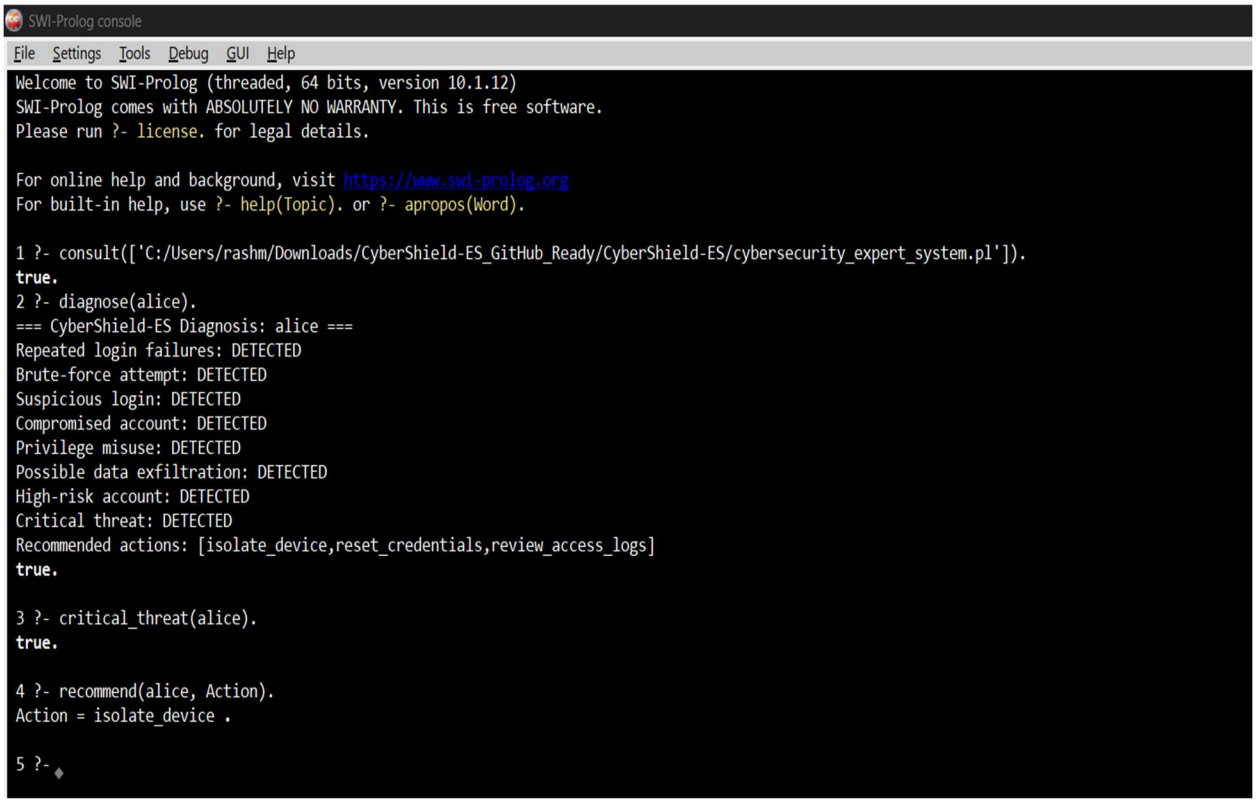

Figure 3: Prolog Backtracking Through Multiple Security Recommendations

7. FORWARD & BACKWARD CHAINING:

Forward Chaining:

Forward chaining is data-driven. The corrected forward_chain/2 predicate begins with symbolic facts derived from the observed evidence. The forward_fixpoint/2 predicate fires all applicable production rules, appends only new conclusions and repeats until closure. For Alice, repeated login failures derive brute_force_attempt; brute force and suspicious login derive compromised_account; compromise and privilege escalation derive high_risk_account; file access and abnormal owned-device traffic derive possible_data_exfiltration; and the final combination derives critical_threat and response actions.

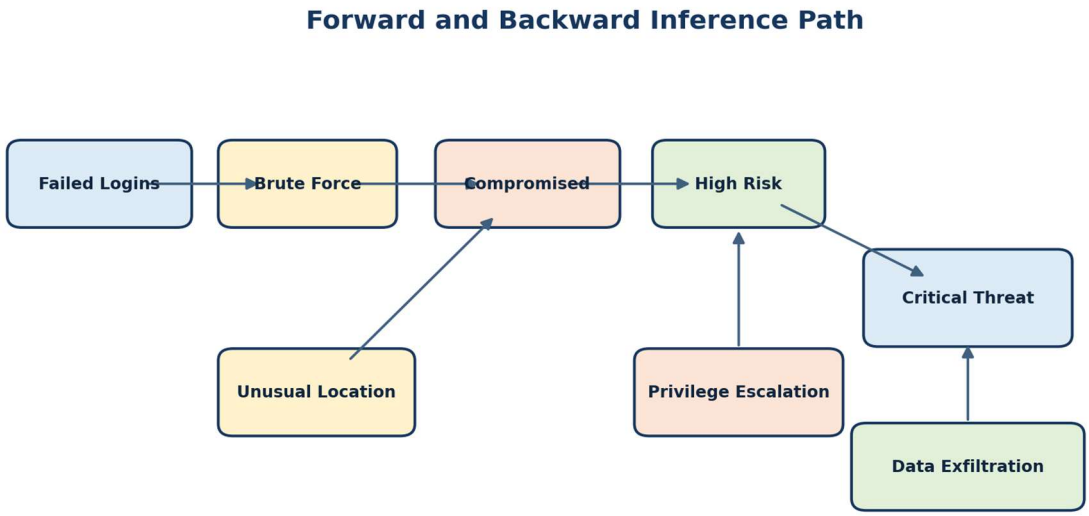

Figure 4: Forward and Backward Inference Path for Cybersecurity Threat Detection

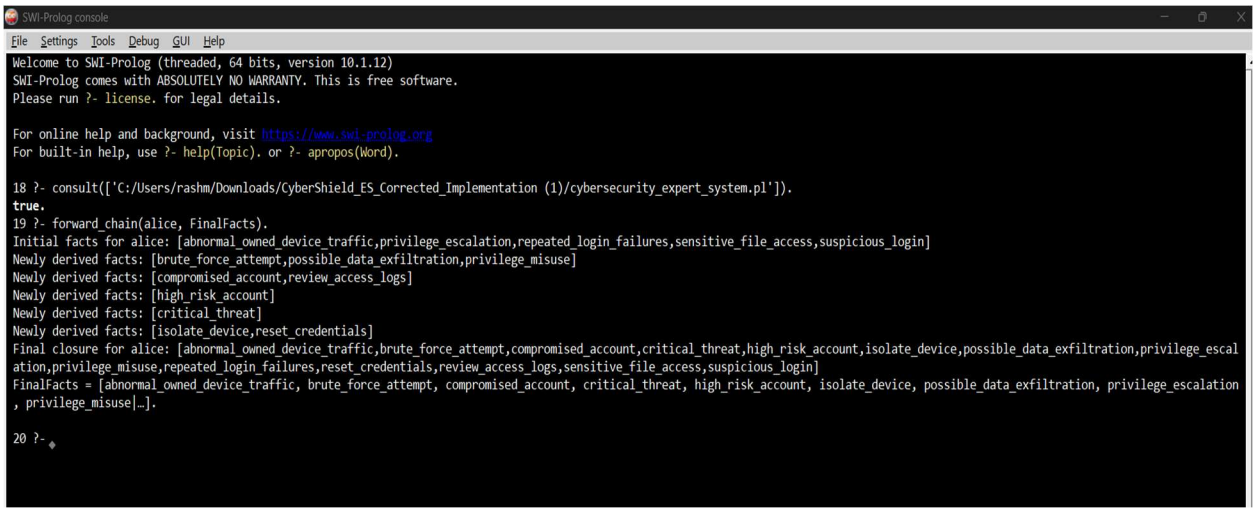

Figure 5: Real SWI-Prolog Forward-Chaining Trace

Backward Chaining:

Backward chaining is goal-driven. For the goal critical_threat(alice), Prolog attempts to prove high_risk_account(alice) and possible_data_exfiltration(alice). It recursively checks account compromise, privilege escalation, brute force, suspicious login, ownership, file access and abnormal traffic. Every subgoal matches the supplied facts, so the critical-threat goal succeeds.

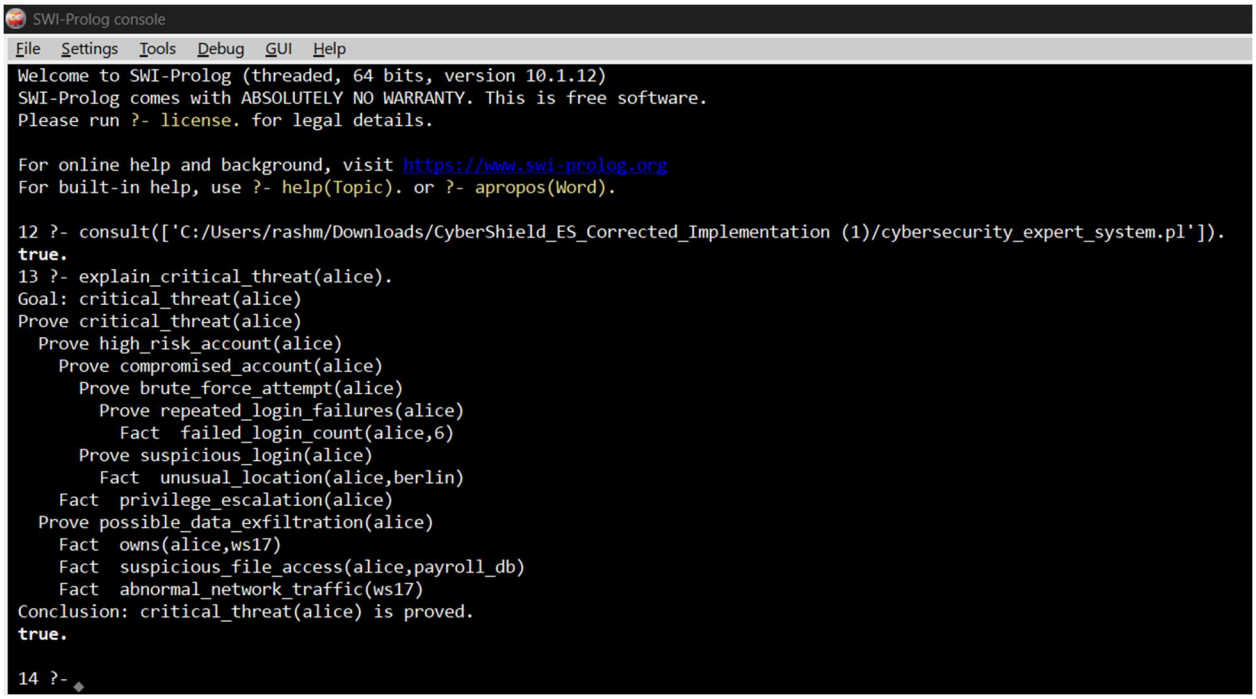

Figure 6: Real SWI-Prolog Recursive Backward-Chaining Trace for Alice

8. COMPARATIVE ANALYSIS:

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Representation	IF-THEN rules	Symbols/formulae	Predicates/quantifiers	Facts and rules
Expressiveness	Moderate	Limited	Very high	High
Inference	Rule firing	Modus ponens	Resolution	SLD resolution

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Forward chaining	Natural	Possible	Possible	Explicitly programmed
Backward chaining	Possible	Proof-based	Theorem proving	Native
Relationships	Moderate	Weak	Excellent	Excellent
Scalability	Good with engines	Poor for many entities	Powerful but complex	Good for bounded systems
Explainability	Very high	High for small cases	High but complex	High
Implementation	Easy	Easy for small cases	Difficult	Easy to moderate
Suitability	Alert generation	Basic logical model	Best formal model	Best executable prototype

Production rules offer operational transparency. Propositional logic is precise but rigid. First-order logic provides the richest formal representation. Prolog offers the best balance of expressiveness, executability and understandable inference for the selected bounded expert system.

Procedural and Non-Procedural Paradigm Comparison:

A procedural implementation specifies the exact sequence of operations: read each event, evaluate every condition, update flags and control the order of execution. The programmer is responsible for loops, branches and state changes. In contrast, the Prolog implementation is declarative or non-procedural: it states facts and logical relationships, while the inference mechanism determines how a goal can be proved through unification and backtracking.

Aspect	Procedural Approach	Declarative Prolog	Application in CyberShield-ES
Program focus	How to perform each step	What facts and relationships are true	The expert specifies cybersecurity knowledge rather than a fixed query sequence
Control flow	Explicit loops and conditions	Controlled by inference and rule matching	Prolog selects matching rules automatically
Reuse	Procedures often depend on fixed order	Rules apply to any matching user	The same diagnosis rules work for Alice, Charlie, David and Bob
Explanation	Must be programmed separately	Proof path follows matched goals	explain_critical_threat/1 exposes the reasoning trace

9. RESULTS, DISCUSSION & RECOMMENDED MODEL:

The knowledge base detected repeated login failures, a brute-force attempt, suspicious login, account compromise, privilege misuse, possible data exfiltration, a high-risk account and a critical threat for Alice. The unsupported query for Bob returned false, confirming that conclusions are generated only when their required evidence can be proved.

```
5 ?- all_recommendations(alice, Actions).
Actions = [isolate_device, reset_credentials, review_access_logs].

6 ?- explain_critical_threat(alice).
Goal: critical_threat(alice)
Checking compromised_account(alice) ... true
Checking privilege_escalation(alice) ... true
Checking possible_data_exfiltration(alice) ... true
Conclusion: critical_threat(alice) is proved.
true.

7 ?- critical_threat(bob).
false.
```

Figure 7: Negative Test Result for an Unsupported User

Test ID	Query	Expected Output	Status
T1	repeated_login_failures(alice).	true	Pass
T2	brute_force_attempt(alice).	true	Pass
T3	suspicious_login(alice).	true	Pass
T4	compromised_account(alice).	true	Pass
T5	privilege_misuse(alice).	true	Pass
T6	possible_data_exfiltration(alice).	true	Pass
T7	high_risk_account(alice).	true	Pass
T8	critical_threat(alice).	true	Pass
T9	critical_threat(bob).	false	Pass
T10	all_recommendations(alice,A).	three actions	Pass
T11	forward_chain(alice,F), memberchk(critical_threat,F).	true	Pass
T12	brute_force_attempt(charlie), \+ compromised_account(charlie).	true	Pass
T13	possible_data_exfiltration(david), \+ critical_threat(david).	true	Pass
T14	\+ repeated_login_failures(bob), \+ critical_threat(bob).	true	Pass

Prolog is recommended because it supports structured facts and rules, reusable variables, relationships, unification, backtracking, native backward chaining, direct queries and easy expansion. Limitations include fixed thresholds, a small predefined dataset, no live network feed and no uncertainty score. Future improvements can add timestamps, adaptive thresholds, confidence values, log-file analysis, a graphical interface and intrusion-detection-system integration.

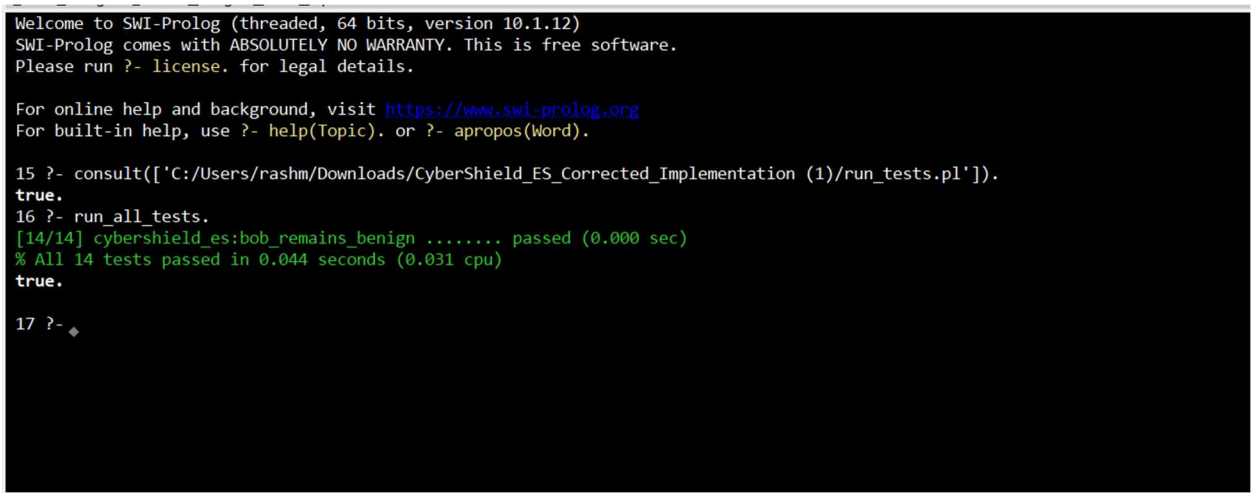

Figure 8: Real SWI-Prolog Results of the Corrected 14-Test Suite

Evaluation of Correctness, Coverage, Consistency and Performance:

Reasoning performance is appropriate for this bounded and acyclic knowledge base because every rule adds a conclusion only once and the forward-chaining engine terminates when it reaches a fixpoint. The complete fourteen-test suite passed in 0.044 seconds, with 0.031 seconds of CPU time, as shown in Figure 8. The prototype remains limited by synthetic facts, a fixed threshold, closed-world reasoning and the absence of confidence scores..

Coverage is improved by testing every major diagnosis predicate, all applicable recommendations, explicit forward chaining, a brute-force-only case, a data-exfiltration-only case and a benign case. Consistency is supported because critical_threat requires both high_risk_account and possible_data_exfiltration, preventing escalation when only one evidence branch is present.

Reasoning performance is appropriate for this bounded acyclic knowledge base because each rule adds a conclusion only once and the forward engine terminates at a fixpoint. No numerical timing claim is made because performance has not yet been measured on the corrected implementation. The prototype remains limited by synthetic facts, a fixed threshold, closed-world reasoning and the absence of confidence scores.

10. CONCLUSION:

This report developed and compared production rules, propositional logic, first-order logic, procedural programming and declarative Prolog for cybersecurity threat detection. The corrected system combines an explicit data-driven forward-chaining engine with Prolog backward chaining, unification, backtracking, explanation and grouped testing.

The original implementation and ten-test suite successfully derived the documented Alice and Bob results. The corrected implementation successfully executed all fourteen test cases in SWI-Prolog. The real outputs confirm explicit forward chaining, backward goal-directed reasoning, unification, backtracking, multiple scenario coverage and correct handling of unsupported conclusions.

11. REFERENCES:

1. Bratko, Ivan. Prolog Programming for Artificial Intelligence. 4th ed. Addison-Wesley, 2012.

2. National Institute of Standards and Technology. The NIST Cybersecurity Framework 2.0. 2024.

3. Russell, Stuart J., and Peter Norvig. Artificial Intelligence: A Modern Approach. 4th ed. Pearson, 2021.

4. SWI-Prolog. “SWI-Prolog Reference Manual.” Accessed August 2026. https://www.swi-prolog.org/pldoc/man?section=manual.

5. MITRE. “MITRE ATT&CK Enterprise Matrix.” Accessed August 2026. https://attack.mitre.org/.

12. GITHUB REPOSITORY:

The complete Prolog program, README file, test queries and output screenshots are available at:

https://github.com/Rashmitha22/MLA0104_AIES/tree/main/ASSESSMENT%20TOOL/C05/AT1

APPENDIX - COMPLETE PROLOG PROGRAM:

```
/*
  CyberShield-ES
  Cybersecurity threat detection using knowledge representation and reasoning.

  Load in SWI-Prolog:
     ?- [cybersecurity_expert_system].
*/

% ----------------------------
% Observed cybersecurity facts
% ----------------------------

failed_login_count(alice, 6).
unusual_location(alice, berlin).
privilege_escalation(alice).
suspicious_file_access(alice, payroll_db).
abnormal_network_traffic(ws17).
owns(alice, ws17).

% Additional scenarios for coverage and negative testing.
failed_login_count(charlie, 7).
owns(charlie, ws21).

failed_login_count(david, 1).
suspicious_file_access(david, customer_db).
abnormal_network_traffic(ws30).
owns(david, ws30).

failed_login_count(bob, 1).
owns(bob, ws10).
```

```
% ----------------------------
% Threat-diagnosis rules
% ----------------------------

repeated_login_failures(User) :-
    failed_login_count(User, Count),
    Count >= 5.

brute_force_attempt(User) :-
    repeated_login_failures(User).

suspicious_login(User) :-
    unusual_location(User, _).

compromised_account(User) :-
    brute_force_attempt(User),
    suspicious_login(User).

privilege_misuse(User) :-
    privilege_escalation(User),
    suspicious_file_access(User, _).

possible_data_exfiltration(User) :-
    owns(User, Device),
    suspicious_file_access(User, _),
    abnormal_network_traffic(Device).

high_risk_account(User) :-
    compromised_account(User),
    privilege_escalation(User).

critical_threat(User) :-
```

```
    high_risk_account(User),
    possible_data_exfiltration(User).

% ----------------------------
% Recommended defensive actions
% ----------------------------

recommend(User, isolate_device) :-
    critical_threat(User).

recommend(User, reset_credentials) :-
    critical_threat(User).

recommend(User, review_access_logs) :-
    privilege_misuse(User).

recommend(User, enforce_mfa) :-
    brute_force_attempt(User),
    \+ compromised_account(User).

all_recommendations(User, Actions) :-
    setof(Action, recommend(User, Action), Actions).

% ----------------------------
% Explicit forward-chaining engine
% ----------------------------

% Observable facts are converted into symbolic working-memory facts.
forward_seed(User, repeated_login_failures) :-
    failed_login_count(User, Count),
    Count >= 5.
```

```
forward_seed(User, suspicious_login) :-
   unusual_location(User, _).

forward_seed(User, privilege_escalation) :-
   privilege_escalation(User).

forward_seed(User, sensitive_file_access) :-
   suspicious_file_access(User, _).

forward_seed(User, abnormal_owned_device_traffic) :-
   owns(User, Device),
   abnormal_network_traffic(Device).

% Production rules used by the forward-chaining engine.
forward_rule(brute_force_attempt,
        [repeated_login_failures]).

forward_rule(compromised_account,
        [brute_force_attempt, suspicious_login]).

forward_rule(privilege_misuse,
        [privilege_escalation, sensitive_file_access]).

forward_rule(possible_data_exfiltration,
        [sensitive_file_access, abnormal_owned_device_traffic]).

forward_rule(high_risk_account,
        [compromised_account, privilege_escalation]).

forward_rule(critical_threat,
        [high_risk_account, possible_data_exfiltration]).
```

```
forward_rule(isolate_device,
        [critical_threat]).

forward_rule(reset_credentials,
        [critical_threat]).

forward_rule(review_access_logs,
        [privilege_misuse]).

% Repeatedly fires every applicable rule until no new fact can be added.
forward_chain(User, FinalFacts) :-
   findall(Fact, forward_seed(User, Fact), SeedFacts0),
   sort(SeedFacts0, SeedFacts),
   format('Initial facts for ~w: ~w~n', [User, SeedFacts]),
   forward_fixpoint(SeedFacts, FinalFacts),
   format('Final closure for ~w: ~w~n', [User, FinalFacts]).

forward_fixpoint(CurrentFacts, FinalFacts) :-
   findall(Conclusion,
         ( forward_rule(Conclusion, Conditions),
           all_present(Conditions, CurrentFacts),
           \+ memberchk(Conclusion, CurrentFacts)
         ),
         NewFacts0),
   sort(NewFacts0, NewFacts),
   ( NewFacts = [] ->
      FinalFacts = CurrentFacts
   ;
      format('Newly derived facts: ~w~n', [NewFacts]),
      append(CurrentFacts, NewFacts, ExpandedFacts0),
      sort(ExpandedFacts0, ExpandedFacts),
      forward_fixpoint(ExpandedFacts, FinalFacts)
```

```
    ).

all_present([], _).
all_present([Condition|Conditions], Facts) :-
    memberchk(Condition, Facts),
    all_present(Conditions, Facts).

% ----------------------------
% Explanation facility
% ----------------------------

explain_critical_threat(User) :-
    format('Goal: critical_threat(~w)~n', [User]),
    trace_goal(critical_threat(User), 0),
    format('Conclusion: critical_threat(~w) is proved.~n', [User]).

trace_goal(critical_threat(User), Depth) :-
    trace_line(Depth, critical_threat(User)),
    Next is Depth + 1,
    trace_goal(high_risk_account(User), Next),
    trace_goal(possible_data_exfiltration(User), Next).

trace_goal(high_risk_account(User), Depth) :-
    trace_line(Depth, high_risk_account(User)),
    Next is Depth + 1,
    trace_goal(compromised_account(User), Next),
    trace_goal(privilege_escalation(User), Next).

trace_goal(compromised_account(User), Depth) :-
    trace_line(Depth, compromised_account(User)),
    Next is Depth + 1,
    trace_goal(brute_force_attempt(User), Next),
```

```
    trace_goal(suspicious_login(User), Next).

trace_goal(brute_force_attempt(User), Depth) :-
    trace_line(Depth, brute_force_attempt(User)),
    Next is Depth + 1,
    trace_goal(repeated_login_failures(User), Next).

trace_goal(repeated_login_failures(User), Depth) :-
    trace_line(Depth, repeated_login_failures(User)),
    Next is Depth + 1,
    failed_login_count(User, Count),
    Count >= 5,
    trace_fact(Next, failed_login_count(User, Count)).

trace_goal(suspicious_login(User), Depth) :-
    trace_line(Depth, suspicious_login(User)),
    Next is Depth + 1,
    unusual_location(User, Location),
    trace_fact(Next, unusual_location(User, Location)).

trace_goal(privilege_escalation(User), Depth) :-
    privilege_escalation(User),
    trace_fact(Depth, privilege_escalation(User)).

trace_goal(possible_data_exfiltration(User), Depth) :-
    trace_line(Depth, possible_data_exfiltration(User)),
    Next is Depth + 1,
    owns(User, Device),
    suspicious_file_access(User, File),
    abnormal_network_traffic(Device),
    trace_fact(Next, owns(User, Device)),
    trace_fact(Next, suspicious_file_access(User, File)),
```

```
    trace_fact(Next, abnormal_network_traffic(Device)).

trace_line(Depth, Goal) :-
    Indent is Depth * 2,
    format('~*cProve ~q~n', [Indent, 32, Goal]).

trace_fact(Depth, Fact) :-
    Indent is Depth * 2,
    format('~*cFact  ~q~n', [Indent, 32, Fact]).

check(Goal) :-
    format('Checking ~q ... ', [Goal]),
    ( call(Goal) ->
        writeln(true)
    ;
        writeln(false),
        fail
    ).

% ----------------------------
% Human-readable diagnosis
% ----------------------------

diagnose(User) :-
    format('=== CyberShield-ES Diagnosis: ~w ===~n', [User]),
    print_result('Repeated login failures', repeated_login_failures(User)),
    print_result('Brute-force attempt', brute_force_attempt(User)),
    print_result('Suspicious login', suspicious_login(User)),
    print_result('Compromised account', compromised_account(User)),
    print_result('Privilege misuse', privilege_misuse(User)),
    print_result('Possible data exfiltration', possible_data_exfiltration(User)),
    print_result('High-risk account', high_risk_account(User)),
```

```
    print_result('Critical threat', critical_threat(User)),
    ( all_recommendations(User, Actions) ->
        format('Recommended actions: ~w~n', [Actions])
    ;
        writeln('Recommended actions: none')
    ).

print_result(Label, Goal) :-
    ( call(Goal) -> Status = 'DETECTED' ; Status = 'NOT DETECTED' ),
    format('~w: ~w~n', [Label, Status]).
```

APPENDIX - TEST QUERIES AND OUTPUTS:

```
?- repeated_login_failures(alice).
true.

?- brute_force_attempt(alice).
true.

?- suspicious_login(alice).
true.

?- compromised_account(alice).
true.

?- privilege_misuse(alice).
true.

?- possible_data_exfiltration(alice).
true.

?- high_risk_account(alice).
```

```
true.
?- critical_threat(alice).
true.
?- recommend(alice, Action).
Action = isolate_device ;
Action = reset_credentials ;
Action = review_access_logs.
?- all_recommendations(alice, Actions).
Actions = [isolate_device, reset_credentials, review_access_logs].
?- critical_threat(bob).
false.
```

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, facts, conditions, goals, and expected conclusions.	7–8% – Good understanding of the domain with minor omissions.	5–6% – Basic domain understanding with several missing elements.	0–4% – Poor or incorrect understanding of the problem domain.
2. Knowledge Acquisition and Representation	10%	9–10% – Acquires relevant domain knowledge and represents it accurately using well-structured Prolog facts and rules.	7–8% – Knowledge is appropriately represented with minor gaps.	5–6% – Basic knowledge representation with missing or unclear facts/rules.	0–4% – Inadequate or inappropriate knowledge representation.
3. Production Rule / Knowledge Base Design	15%	13–15% – Develops a comprehensive, consistent, and logically structured production-rule knowledge base with appropriate facts and rules.	10–12% – Well-designed knowledge base with minor inconsistencies or omissions.	7–9% – Basic rule base developed but several rules are incomplete or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
4. Prolog Implementation	15%	13–15% – Implements a fully functional expert system using Prolog facts, predicates, rules, variables, unification, and backtracking effectively.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.

5. Forward Chaining	10%	9–10% – Correctly implements and demonstrates forward chaining from initial facts through applicable rules to final conclusions.	7–8% – Forward chaining works correctly with minor limitations.	5–6% – Basic forward reasoning demonstrated but with incomplete rule processing.	0–4% – Forward chaining is missing or incorrectly implemented.
6. Backward Chaining	10%	9–10% – Effectively demonstrates goal-driven backward chaining using Prolog's inference mechanism and clearly traces the reasoning process.	7–8% – Backward chaining works correctly with minor gaps.	5–6% – Basic backward reasoning demonstrated with limited explanation.	0–4% – Backward chaining is missing or incorrectly implemented.
7. Procedural and Non-Procedural Paradigm Analysis	10%	9–10% – Clearly distinguishes procedural and declarative/non-procedural approaches and demonstrates their application in the Prolog system.	7–8% – Provides a good distinction with minor conceptual gaps.	5–6% – Basic distinction is explained but lacks practical demonstration.	0–4% – Paradigms are misunderstood or not addressed.
8. Inference, Unification and Reasoning Accuracy	10%	9–10% – Produces accurate conclusions using unification, backtracking, and logical inference across diverse test cases.	7–8% – Reasoning is mostly accurate with minor errors.	5–6% – Basic inference works but produces occasional incorrect results.	0–4% – Inference is unreliable or incorrect.

9. Testing, Results and System Demonstration	5%	5% – Provides comprehensive test cases, accurate outputs, reasoning traces, and a convincing live demonstration.	4% – Good testing and demonstration with minor gaps.	2–3% – Limited test cases and basic demonstration.	0–1% – Testing or demonstration is missing/inadequate.
10. Documentation, GitHub and Technical Presentation	5%	5% – Complete documentation, well-organized GitHub repository, clear code, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation/repository with several omissions.	0–1% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				